

Liana is a Domain-Specific Language (DSL) designed for the formal specification of **Timed Automata**. Unlike graphical tools, Liana provides a strict, textual, and declarative structure that maps directly to the mathematical 5-tuple definition of an automaton (States, Actions, Clocks, Transitions, and Initial conditions).

The language is divided into a **Structural Header** (defining the environment) and a **Behavioral Body** (defining the logic).

1. Structural Header: The Environment

Every Liana program begins by declaring the global resources the automaton will use.

- **Clocks:** Declared in the `clocks` block. These are continuous variables that increase at the same rate as time.
- **Actions:** Declared in the `actions` block. These represent synchronization labels for communication between different automata.

- **Integers (Optional):** Declared in the `integers` block. Unlike clocks, these are discrete variables. According to the grammar, they are automatically initialized to zero.
 - **Symmetry:** An automaton can be marked with `:: symm <int>`. This indicates it belongs to a specific symmetry group, meaning other automata in the same group are structurally identical, which is used to optimize formal verification.
-

2. Locations: Defining States

The `locations` block defines the nodes of the graph. Each location (state) can have a set of properties wrapped in angle brackets `< >`.

- **Initial State (`ini`):** Marked with `ini: T` (True) or `ini: F` (False). This determines where the execution starts.
- **Urgency (`urg`):** A location marked with `urg: T` is an "urgent" state. Time is not allowed to progress while the automaton is in this state; a transition must be taken immediately.

- **Invariants (`inv`):** These are clock constraints (e.g., `[ (x, <=, 5) ]`) that must hold true as long as the automaton stays in that location. If time progression would violate the invariant, the automaton *must* leave the state via a transition.
-

3. Transitions: Defining Logic

Transitions are the edges of the graph and are defined as explicit tuples. A transition rule follows this specific order:

```
(source_location, action, clock_guards, [boolean_guards], [clock_resets],  
[assignments], target_location)
```

Components of a Transition:

- **Action Label:** Uses `!` for output/sending and `?` for input/receiving (e.g., `switch_on!`).
- **Clock Guards:** A list of constraints on clock variables. A transition can only fire if the current clock values satisfy these conditions (e.g., `[(x, ==, 2)]`).

- **Boolean Guards (Optional):** Additional logical conditions involving integer variables (e.g., `y > 10 && z == 0`).
 - **Clock Resets:** A list of clocks that are reset to zero when the transition fires, denoted in square brackets `[x, y]` .
 - **Assignments (Optional):** Update expressions for integer variables (e.g., `[count = count + 1]`).
-

4. Expressions and Constraints

Liana provides a full grammar for both arithmetic and boolean logic to support complex guards and assignments.

- **Punctual Guards:** Liana supports the `==` operator. In Timed Automata theory, a guard like `(x, ==, 2)` means the transition is enabled *only* at that exact moment.

- **Disjunction (OR):** The grammar explicitly states that clock guards cannot express disjunctions. To model an `OR` condition, you must define multiple separate transitions between the same states, one for each disjunct.
 - **Boolean Logic:** Supports standard operators: `&&` (AND), `||` (OR), and comparison operators (`<`, `<=`, `==`, `>=`, `>`).
 - **Arithmetic:** Supports basic operations: `+`, `-`, `*`, `/`, and parentheses for grouping.
-

5. Semantic Mapping Example

If you have a transition defined as:

```
(off, switch_on!, [(x, >=, 0)], [x], on)
```

It interprets semantically as:

1. **Source:** The automaton starts in state `off`.
2. **Trigger:** It sends the `switch_on` signal.
3. **Condition:** It can only do this if clock `x` is greater than or equal to 0.

4. **Action:** Upon firing, clock `x` is reset to 0.

5. **Target:** The automaton moves to state `on`.